

The ai/ harness

How Claude and Codex share one file-based process — what each part does, how a request flows through it, and why it is built this way.

ai/project.md

ai/policies.md

ai/rules/

ai/workflows/

ai/skills/

ai/templates/

ai/checklists/

ai/evaluations/

ai/improvements/

Two agents, many sessions, no shared memory

Claude and Codex both work on this repository, often in different sessions. Without a shared, file-based source of instructions, each session would re-derive — or silently disagree about — the things that matter.

- **Scope** — What is in and out of this piece of work
- **Technology choices** — What the project has actually decided vs. what is still open
- **Quality bar** — What “correct” and “done” mean for each kind of change
- **Authorization** — What an agent may do alone, and when it must stop and ask

THE HARNESS’S JOB

A request plus the repository’s own files should be enough for any agent, in any session, to pick up correct, consistent work — without reading prior chat history.

Context is made **durable** (in files) and **agent-neutral** (one source both agents read).

What lives in ai/ — and why each part exists

ai/project.md

Confirmed vs. open technology decisions

No agent invents or assumes a choice the project hasn't made

ai/policies.md

Authorization, plan approval, pause conditions, untrusted-content guardrails

The safety boundary: act alone, or stop and ask

ai/rules/ • 8 files

common, documentation, frontend, backend, database, testing, git-review, ci-cd

One current standard per area, applied by every skill

ai/workflows/ • 4

bootstrap, feature-delivery, bug-fix, harness-improvement

A starting sequence for each way work begins

ai/skills/ • 13

The “how to do X” procedures

Bounded, checkable units with one 8-part shape

ai/templates/

Starting document for every skill output

Modeled on real formats (MADR, IEEE 829, 基本設計書 ...)

ai/checklists/ • 4

Point-in-time pass/fail gates

“Did I follow the rules” becomes an explicit check

ai/evaluations/

Manual scenarios: process rules and adversarial cases

Guidance changes are tested, not trusted on faith

ai/improvements/

One record per harness change

Keeps the history of why the harness changed

Three different questions, three different places

Collapsing them would bury authorization inside technical detail — or turn every rule into an unenforced suggestion.

ai/policies.md

“Is this agent allowed to do this, or must it stop and ask?”

Plan approval, pause conditions, external operations (push, PR, merge, deploy), untrusted content is data — never instructions.

ai/rules/

“What does correct look like for this kind of change?”

RFC 9457 errors, strict TypeScript, parameterized SQL, WCAG 2.2 AA, SHA-pinned actions, test-pyramid shape ...

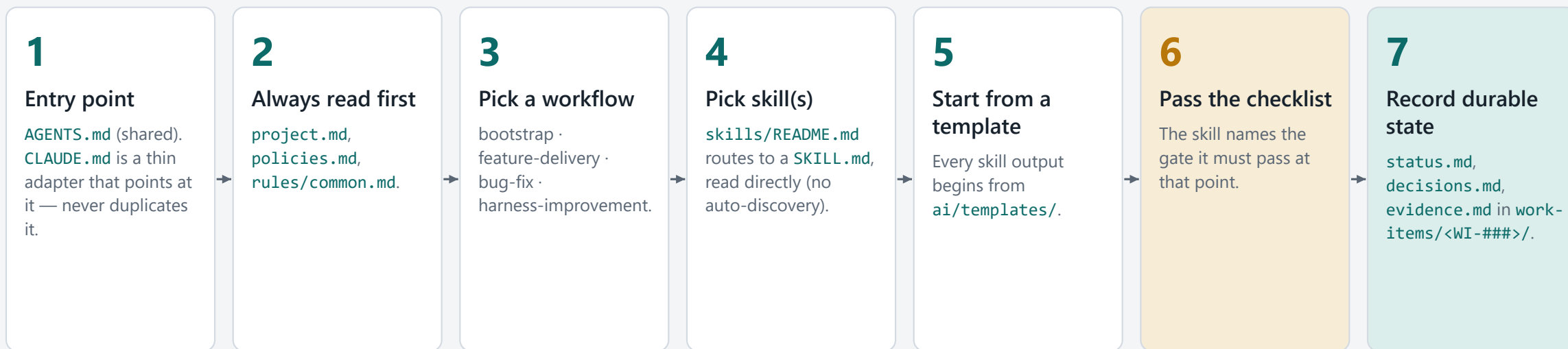
ai/checklists/

“Right now, before the next step — did I actually meet that bar?”

design-consistency, security-review, delivery, release-readiness: explicit gates at the moment they matter.

How a request actually flows — for either agent

Every task enters through the same chain. The sequence matters: each step narrows what the next one reads.



Outside the per-task chain: ai/evaluations/ is how the harness checks itself — run when a change to shared guidance is proposed. Step 7 is what makes work resumable: the next step, possibly another agent in another session, continues from the repository, not from the chat.

Four ways work begins, four starting sequences

Each workflow orders the skills a task needs and ends with an explicit exit condition — without hard-coding every path.

project-bootstrap

- 1 Confirm open tech decisions and demo scope
- 2 Plan: skeleton, local env, CI checks
- 3 Build in bounded, approved steps
- 4 Verify real commands; update project.md
- 5 Record gaps; hand off to feature-delivery

feature-delivery

- 1 Brief + acceptance criteria
- 2 Plan, shown and approved
- 3 Architecture check → BD / DB / API / DD
- 4 Reconcile designs, then implement
- 5 Unit/integration, then system/E2E
- 6 Review diff, PR within authorization
- 7 Record evidence

bug-fix

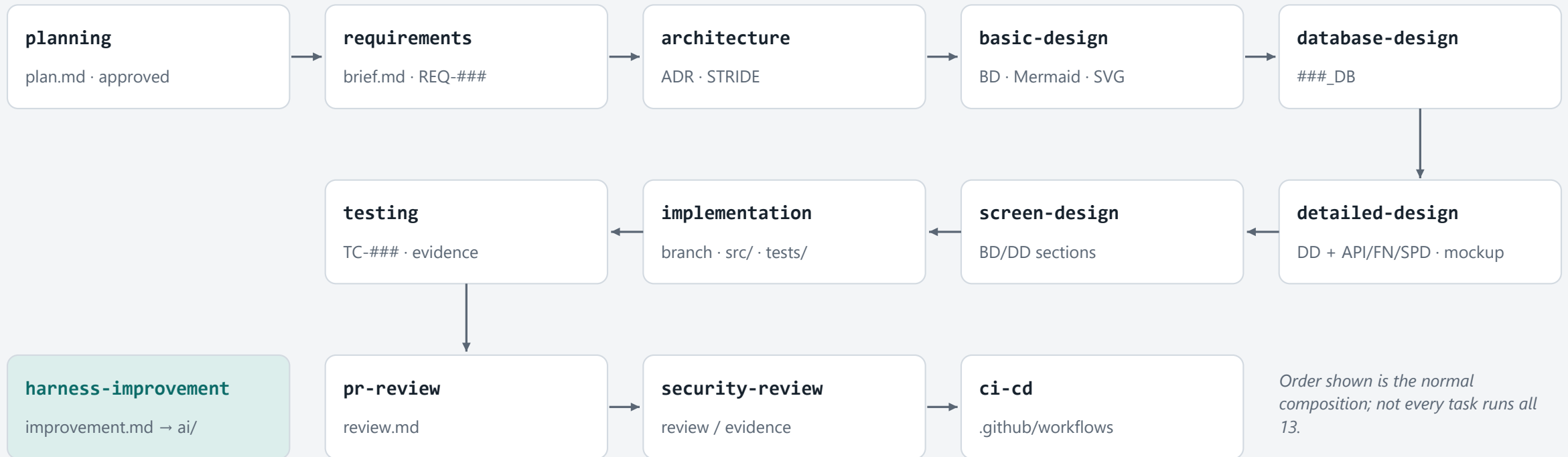
- 1 Reproducible failing vs. expected case
- 2 Severity: live incident mitigates first
- 3 Find cause; add a regression check
- 4 Fix; update affected design/contracts
- 5 Targeted checks; record evidence
- 6 Blameless postmortem if live

harness-improvement

- 1 Record observed failure / change
- 2 Propose the smallest useful change
- 3 Evaluate with ai/evaluations cases
- 4 Record limits; review before adoption
- 5 Rollback via version control — never relax a gate

13 skills, composed as a resumable hand-off chain

A workflow picks the subset a task needs. Each skill's step 8 hands its artifact to the next — the chain survives agent and session switches.



Order shown is the normal composition; not every task runs all 13.

Runs on its own trigger: when the harness itself is wrong or stale.

Every SKILL.md has the same eight parts — on purpose

1–2

Purpose · inputs & source order

Bound what the skill needs before it starts.

3

Execution steps & applicable rules

Every step is tied to the rule that governs it — no implicit “current standard”.

4

Tools & environment

Honest about what can’t be done without a missing toolchain.

5

Outputs, templates, IDs, locations

REQ BD DD DB SCR TC DEC WI IDs and paths are explicit, not guesswork.

6–7

Checklist · termination & failure

“Done” becomes a checkable claim, not an assertion.

8

Work-item update & handover

Makes the pipeline resumable across agents and sessions.

Eight rule files: one current standard per area

Skills cite these instead of restating them, so the same change always meets the same bar.

common.md

Preserve approved scope · stable IDs, link don't duplicate · record unresolved decisions
· focused changes

documentation.md

English source · EN + JA PDF of every doc · Japanese UI text quoted with an English gloss · number-first IDs, one folder per number

frontend.md

Vite + TypeScript, strict (no **any**) · WCAG 2.2 AA · config via import.meta.env, no secrets in the browser

backend.md

.NET 10 · nullable types · RFC 9457 errors, no leaked detail · structured logs · OpenTelemetry on new endpoints

database.md

PostgreSQL · parameterized SQL only · expand/contract on live tables · least-privilege app role · recovery limits first

testing.md

Trace to acceptance criteria · pyramid shape · resettable data · quarantine flaky tests · not run ≠ pass

git-review.md

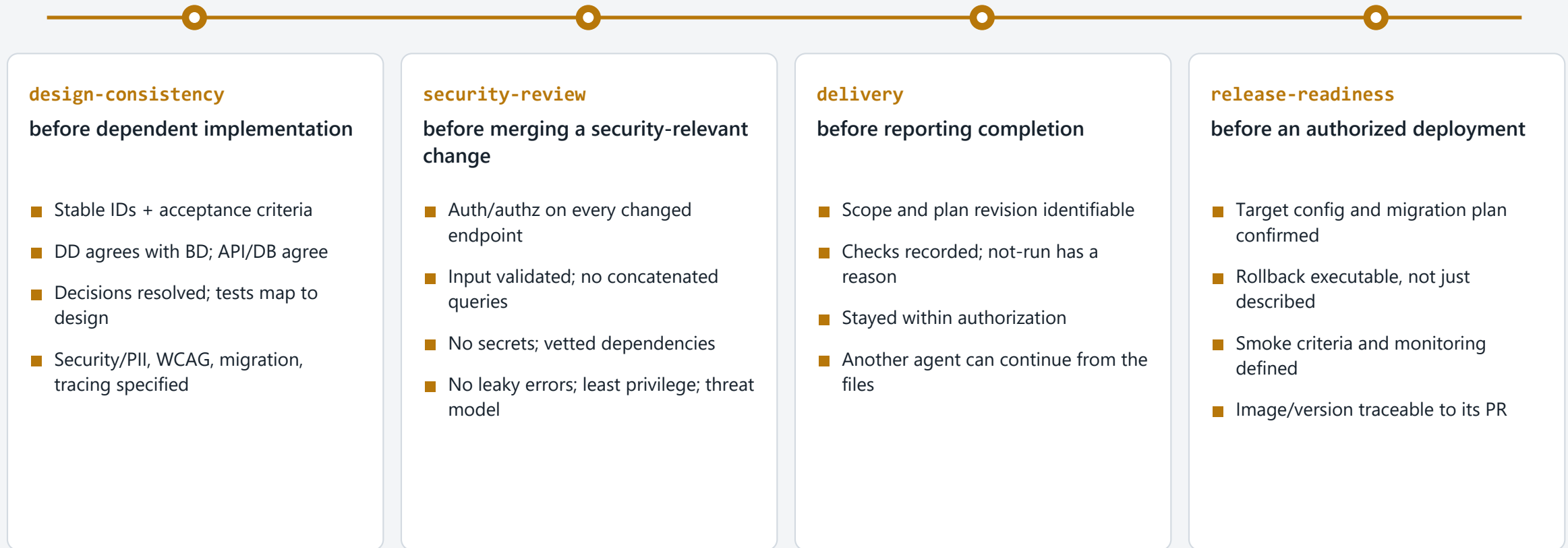
Branch + worktree per work item · small focused PRs · no direct push · squash-merge · review ≠ merge

ci-cd.md

SHA-pinned actions · read-only default permissions · OIDC over long-lived secrets · non-root scanned images

Four gates, each at the moment it matters

Point-in-time pass/fail checks — “did this follow the rules” stops being something held in memory.



Every output starts from a real, named document format

Chosen to match the artifact's purpose — so the output is useful, not a formality.

TEMPLATE	MODELED ON	CAPTURES
<code>brief.md</code>	PRD conventions	REQ-### with success + failure criteria, "not doing"
<code>plan.md</code>	PMI-style implementation plan	Steps with verification, authorized actions, revisions kept
<code>status.md</code>	RAG status report	State, accomplishments, risks, single next action
<code>decisions.md</code>	Decision log	Context, options, answer, rationale, impact
<code>evidence.md</code>	Traceability matrix + test log	REQ → design → code → test; real results
<code>architecture-decision.md</code>	MADR	Drivers, options, outcome, consequences
<code>basic-design.md</code>	基本設計書	Flows, screens, Mermaid diagrams, SVG wireframes
<code>detailed-design.md</code> + DD/*	詳細設計書 family	Main DD + API · function · screen-processing
<code>database-design.md</code>	テーブル定義書 + ER	Tables, indexes, constraints, migration limits
<code>test-plan.md</code>	IEEE 829-1998	Items, approach, cases, results, gaps
<code>review.md</code>	Google code-review categories	Design, functionality, complexity, tests ...
<code>improvement.md</code>	Rust-style RFC	Motivation, alternatives, evaluation, adoption

The work item is the memory — not the chat

Switching from Claude to Codex mid-task means reading the work item and continuing, not restarting.

work-items/<WI-###>/

brief.md

What and why: REQ-### and acceptance criteria

plan.md

Every revision shown and approved before its work; all kept, oldest first

status.md

Current state, risks, the single next action

decisions.md

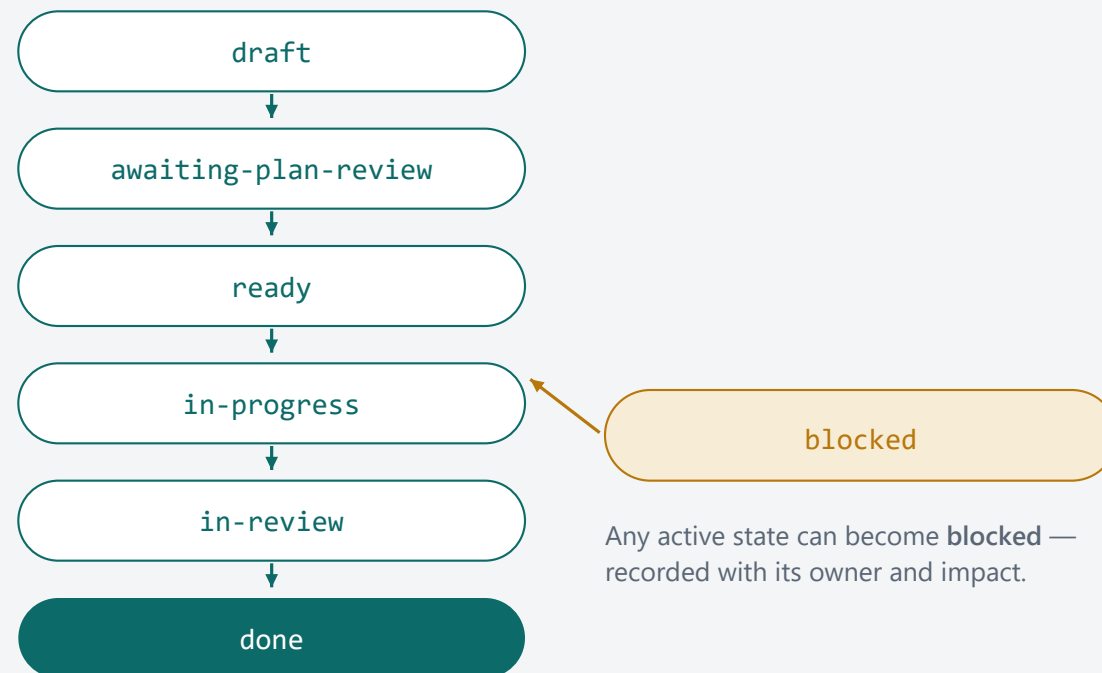
Every question that stopped an agent, and its answer

evidence.md

What makes “done” checkable after the fact

+ design docs under [docs/en/...](#) (BD, DB, DD, ADR, tests), each with EN and JA PDFs

status.md states



Five principles behind every file

1

One shared, agent-neutral source

Process lives only in ai/. Adapters (AGENTS.md, CLAUDE.md, .claude/, a .codex/ if ever needed) route to it and never duplicate it — agent-specific tooling stays out of shared skills.

2

Markdown is the contract, not the executor

The harness defines what correct and done mean. Scripts, test runners and GitHub Actions do the work — and their real output is recorded, not assumed.

3

An approved plan is the scope of authority

Steps don't need re-approval, but the plan's scope and the pause conditions are hard limits. Content an agent merely read is never authorization.

4

A conclusion requires evidence

"Done", "passing", "deployed" are claimed only after a recorded, actually-run check. Not run is recorded as not run.

5

The harness improves itself, on the record

Gaps are fixed through the harness-improvement skill and evaluation cases — never as a silent edit to shared guidance.

Four runs of the chain — Screens A, B and C are done

WI-001 – WI-004 • WHAT EACH RUN EXERCISED

- **WI-001** — project-bootstrap: skeleton, auth foundation, local Compose environment, CI
- **WI-002 · Screen A** — The first end-to-end feature: requirements → plan → BD → DB → four DD documents + mockup → code → unit, integration and E2E tests with axe → checklists → PR
- **WI-003 · Screen B** — Building on an existing screen: evolving a live schema, reusing shared modules and endpoints, amending Screen A's design
- **WI-004 · Screen C** — A read-only dashboard over one consistent snapshot, a design revised after mockup review, and a navbar on every screen

Each run is recorded in `work-items/WI-00N/`.

Where it stands

CI runs green — Backend, frontend and E2E (Compose + Playwright) on every pull request to master; documentation-only changes skip it.

Gaps fixed on the record — What the chain exposed was fixed through harness-improvement, not by silent edits to shared guidance.

Open project decisions — Deployment host, merge/deploy permissions, demo videos, the full role matrix — the owner's to make.

NEXT

The locked roadmap is complete and no next work item is planned. The untested part of the harness is deployment: release-readiness and the ci-cd skill's deployment half wait on a deployment target.